

Exceptions

Exceptions refer to problems in code; errors

Syntax error

- problem with the code that has been written by the user

Unterminated

- Started something but didn't stop it

Syntax errors are problems that the user has to go back and solve in order to fix the code/program.

Runtime errors

- other errors that occur while the code is running

To combat runtime errors, the user/programmer must combat the probabilities of something that can happen.

ex.

```
x = int(input("What's x? "))
print(f"x is {x}")
```

← immediately casting a str into an int would mean the user is confident in the expectation that the inputted value is a number.

if input is not a number, a runtime error occurs.

ex.

```
x = int(input("What's x? "))
print(f"x is {x}")
```

What's x? Poop
ValueError: invalid literal

Value Error

- the inputted or coded value is not a preferred or suitable datatype for the purpose of executing a program.

else

- a clause associated with the exception functions, specifically the "try" function

ex. While True:

```
    try:
        x = int(input("What's X?"))
    except ValueError:
        print("X is not an int")
    else:
        break
```

Print(f"X is {x}")

```
What's X? [cat]
X is not an int
What's X? [dog]
X is not an int
What's X? [bird]
X is not an int
What's X? 4
X is 4
```

If you want to handle an exception in python but not really do anything, essentially ignoring it, use Pass:

ex. While True:

```
    try:
        x = int(input("What's X?"))
    except ValueError:
        pass
    else:
        break
    print(f"X is {x}")
```

```
What's X? [cat]
What's X? [dog]
What's X? [bird]
What's X? 50
X is 50
```